

开源软件基础

Python 动态分析

Zhilei Ren



OSCAR Team, School of Software, Dalian University of Technology

November 17, 2022



Outline

1 inspect

2 sys.settrace

Python 的作用域

Built-in (Python)

Names preassigned in the built-in names module: open, range, SyntaxError....

Global (module)

Names assigned at the top-level of module file, or declared global in a def within the file.

Enclosing function locals

Names in the local scope of any and all enclosing functions (def or lambda), from inner to outer.

Local (function)

Names assigned in any way within a function (def or lambda), and not declared global in that function.

inspect

```
>>> import inspect
>>> def foo():
...     x = 10
...     return inspect.currentframe()
>>> f = foo()
>>> dir(foo)
['__class__',
...
'f_back',
'f_builtins',
'f_code',
'f_globals',
'f_lineno',
'f_locals',
'f_trace']
```

inspect

Table 1: frame 成员

成员	描述
f_back	next outer frame object (this frame's caller)
f_builtins	builtins namespace seen by this frame
f_code	code object being executed in this frame
f_globals	global namespace seen by this frame
f_lasti	index of last attempted instruction in bytecode
f_lineno	current line number in Python source code
f_locals	local namespace seen by this frame
f_trace	tracing function for this frame, or None

inspect

```
x = 10
def outer():
    x = 20
    def inner():
        x = 30
        print(x)
        return inspect.currentframe()
    return inner
inn = outer()
frame = inn()
print(frame.f_locals['x'])
print(frame.f_globals['x'])
print(inspect.getclosurevars(inn))
```

inspect

```
x = 10
def outer():
    x = 20
    def inner():
        # x = 30
        print(x)
        return inspect.currentframe()
    return inner
inn = outer()
frame = inn()
print(frame.f_locals['x'])
print(frame.f_globals['x'])
print(inspect.getclosurevars(inn))
```

Outline

1 inspect

2 sys.settrace

settrace

```
from sys import settrace

def tracer(frame, event, arg = None):
    code = frame.f_code
    func_name = code.co_name
    line_no = frame.f_lineno
    print(f"事件: {event}, 函数: {func_name}(), 行号: {line_no}")
    return tracer

def foo():
    print("in foo")

def bar():
    foo()

old_tracer = settrace(tracer)
bar()
settrace(old_tracer)
```